Week 5: Data Structures: Dictionaries & Sets

Learning Outcome/Trait Assessment:

- Organize data using key-value pairs (dictionaries) for efficient data retrieval.
- Manage unique collections of data using sets and perform set operations.
- Choose the most appropriate data structure (list, tuple, dictionary, or set) for a given problem.
- **Trait Assessment:** Efficient data retrieval, handling unique collections, understanding the trade-offs between different data structures, logical problem-solving using appropriate structures.

Detailed Lecture Topics:

1. **Dictionaries (dict): The Key-Value Pair Store**
    - **Definition:** An unordered, mutable collection of key-value pairs. Keys must be unique and immutable (e.g., strings, numbers, tuples). Values can[1] be of any type.
    - **Creating Dictionaries:**
        - Empty dictionary: my_dict = {} or my_dict = dict()
        - With initial data: student = {"name": "Alice", "age": 20, "major": "CS"}
        - Using dict() with keyword arguments or list of tuples.
    - **Accessing Values:**
        - Using keys: student["name"] (will raise KeyError if key not found).
        - Using get() method: student.get("name"), student.get("city", "Not Available") (returns None or a default value if key not found).
    - **Adding and Modifying Elements:**
        - dictionary[new_key] = new_value: Adds a new key-value pair.
        - dictionary[existing_key] = new_value: Modifies an existing value.
    - **Removing Elements:**
        - del dictionary[key]: Deletes the item with the specified key.
        - pop(key): Removes the item with the specified key and returns its value.
        - popitem(): Removes and returns a random (or last inserted in Python 3.7+) key-value pair.
        - clear(): Empties the dictionary.
    - **Iterating Through Dictionaries:**
        - Iterating over keys (default): for key in dictionary:
        - Iterating over values: for value in dictionary.values():
        - Iterating over key-value pairs: for key, value in dictionary.items():
    - **Other Useful Dictionary Methods:**
        - keys(): Returns a view object of all keys.
        - values(): Returns a view object of all values.

        - items(): Returns a view object of all key-value pairs.[2]
        - len(): Returns the number of items.
        - in operator: key in dictionary for checking key existence.
    - Nested Dictionaries (brief overview of dictionaries containing other dictionaries).
2. **Sets (set): The Unique, Unordered Collection**
    - **Definition:** An unordered, mutable collection of *unique* elements. Duplicates are automatically removed.
    - **Creating Sets:**
        - From a list/tuple: my_set = set([1, 2, 2, 3]) -> {1, 2, 3}
        - With elements: unique_nums = {1, 2, 3}
        - **Important:** empty_set = set() (because {} creates an empty dictionary).
    - **Adding and Removing Elements:**
        - add(item): Adds a single element.
        - remove(item): Removes an item (raises KeyError if not found).
        - discard(item): Removes an item if present (does not raise error if not found).
        - pop(): Removes and returns an arbitrary element (sets are unordered).
        - clear(): Empties the set.
    - **Set Operations (Mathematical Set Theory):**
        - union() or |: All unique elements from both sets.
        - intersection() or &: Common elements in both sets.
        - difference() or -: Elements in the first set but not in the second.
        - symmetric_difference() or ^: Elements in either set, but not in both.
        - issubset(), issuperset(), isdisjoint().
    - **Use Cases:** Removing duplicates from a list, checking membership efficiently, performing mathematical set operations.
3. **Choosing the Right Data Structure:**
    - **List:** Ordered, mutable, allows duplicates, good for sequences where order matters and elements might change.
    - **Tuple:** Ordered, immutable, allows duplicates, good for fixed collections of related items (e.g., coordinates).
    - **Dictionary:** Unordered (pre-Python 3.7), mutable, key-value pairs, efficient lookups by key, good for representing structured records or mapping relationships.
    - **Set:** Unordered, mutable, *unique* elements, good for membership testing, removing duplicates, and mathematical set operations.

Detailed Labs:

- **Lab 5.1: Simple Contact Book Manager (Dictionary Focus)**
    - **Objective:** Students will create a text-based contact book where they can add, view, update, and delete contacts using a dictionary.
    - **Tasks (Common for Local & Colab):**

1. Create a new Python file (lab5_1.py) or Colab code cell.
2. Initialize an empty dictionary: contacts = {}.
3. Implement a while loop that presents a menu to the user:
    - 1. Add/Update Contact
    - 2. View Contact Details
    - 3. List All Contacts
    - 4. Delete Contact
    - 5. Exit
4. **Add/Update:** If user enters an existing name, update their info. If new, add. Store contact name as key, and a dictionary of details (e.g., {"phone": "123-4567", "email": "a@b.com"}) as value.
5. **View:** Prompt for a contact name, use get() to retrieve and print details, or "Contact not found."
6. **List All:** Iterate through contacts.items() and print each contact's name and details.
7. **Delete:** Prompt for a contact name, use pop() or del to remove it. Handle "Contact not found."
8. **Challenge:** Ensure keys are case-insensitive when searching/deleting (convert user input to .lower() or .title() consistently).

- **Lab 5.2: Course Enrollment & Collaboration (Set Focus)**
    - **Objective:** Students will simulate course enrollments and find common students using set operations.
    - **Tasks (Common for Local & Colab):**
        1. Create a new Python file (lab5_2.py) or Colab code cell.
        2. Define two sets of student IDs for two different courses:
            - python_students = {"S001", "S003", "S005", "S007", "S009"}
            - java_students = {"S002", "S003", "S006", "S007", "S010"}
        3. **Find Students in Both Courses:** Use the intersection() method or & operator to find students enrolled in *both* Python and Java. Print the result.
        4. **Find All Unique Students:** Use the union() method or | operator to find all unique students across both courses. Print the result.
        5. **Find Students Only in Python:** Use the difference() method or - operator to find students in Python but *not* in Java. Print the result.
        6. **Find Students in Only One Course (Exclusive):** Use the symmetric_difference() method or ^ operator to find students enrolled in Python *or* Java, but *not* both. Print the result.

7. **Remove Duplicates from a List:** Create a list with duplicate student IDs (e.g., all_students_list = ["S001", "S003", "S005", "S001", "S002"]). Convert it to a set to remove duplicates, then convert it back to a list. Print the original and the unique list.

Detailed Activities:

- **Activity 5.1: Real-World Applications of Dictionaries and Sets**
  - **Format:** Brainstorming session (small groups or full class).
  - **Instructions:** Ask students to identify real-world scenarios or problems where Dictionaries and Sets would be the ideal data structures to use, explaining why.
  - **Prompts:**
    - Where would you use a dictionary in a game? (e.g., player inventory, character stats)
    - How could a dictionary represent data about a product in an online store? (e.g., product details by ID)
    - When would a set be useful in managing user permissions or tags? (e.g., unique roles, unique hashtags)
    - Can you think of a scenario where you'd need to quickly check if an item exists in a large collection without caring about its order? (e.g., blocked IP addresses, unique visitors).
  - **Purpose:** Solidify understanding of practical applications and encourage critical thinking about data structure choices.
- **Activity 5.2: Word Frequency Counter (using Dictionary)**
  - **Format:** Individual coding challenge.
  - **Instructions:** Students will write a program that takes a sentence from the user and counts the frequency of each word using a dictionary.
  - **Steps:**
    1. Prompt the user to enter a sentence.
    2. Convert the sentence to lowercase: sentence.lower().
    3. Split the sentence into words: words = sentence.split().
    4. Initialize an empty dictionary: word_counts = {}.
    5. Loop through each word in words.
    6. Inside the loop, use if word in word_counts: to check if the word is already a key.
       - If yes, increment its count: word_counts[word] += 1.
       - If no, add it to the dictionary with a count of 1: word_counts[word] = 1.
    7. Print the word_counts dictionary.
  - **Challenge:** Handle punctuation (e.g., remove commas, periods before counting words). *Hint: word.strip('.,!?"')*
  - **Purpose:** Apply dictionary operations to a common text processing task, demonstrating efficient counting and retrieval.

Sample Snippets for Class and Colab Ready

1. Dictionaries: Creation, Access, Modification, Iteration

VS Code (dict_operations.py):

Python

```
# dict_operations.py

print("--- Dictionary Creation and Access ---")
# Creating a dictionary
student_profile = {
    "name": "John Doe",
    "age": 21,
    "major": "Computer Science",
    "gpa": 3.7,
    "courses": ["Python 101", "Data Structures", "Calculus I"]
}
print(f"Student Profile: {student_profile}")

# Accessing values
print(f"Student's Name: {student_profile['name']}")
print(f"Student's Age: {student_profile.get('age')}") # Using get()
print(f"Student's City (using get with default): {student_profile.get('city', 'Not Specified')}")

# Accessing a non-existent key will raise KeyError if not using .get()
# print(student_profile['city']) # Uncomment to see error

print("\n--- Adding and Modifying Dictionary Elements ---")
# Adding a new key-value pair
student_profile["city"] = "Bangkok"
print(f"After adding city: {student_profile}")

# Modifying an existing value
student_profile["gpa"] = 3.9
print(f"After updating GPA: {student_profile}")
```

```
# Using a variable as a key
favorite_language = "favorite_lang"
student_profile[favorite_language] = "Python"
print(f"After adding favorite language: {student_profile}")

print("\n--- Removing Elements from Dictionary ---")
# Using del
del student_profile["courses"]
print(f"After deleting 'courses': {student_profile}")

# Using pop()
removed_major = student_profile.pop("major")
print(f"Removed major: {removed_major}")
print(f"After popping 'major': {student_profile}")

# Using popitem() (removes last inserted in Python 3.7+ or arbitrary)
random_item = student_profile.popitem()
print(f"Removed random item: {random_item}")
print(f"After popitem: {student_profile}")

# student_profile.clear() # Empties the dictionary
# print(f"After clear: {student_profile}")

print("\n--- Iterating Through Dictionaries ---")
my_book = {
    "title": "The Python Journey",
    "author": "A. Coder",
    "pages": 350,
    "published": 2023
}

print("Iterating over keys:")
for key in my_book: # Default iteration is over keys
    print(key)

print("\nIterating over values:")
for value in my_book.values():
    print(value)

print("\nIterating over key-value pairs (items):")
for key, value in my_book.items():
```

```
    print(f"{key}: {value}")

print("\n--- Checking for key existence ---")
if "author" in my_book:
    print("Author exists in the book dictionary.")
```

Google Colab (in a code cell):

```
    Python


# dict_operations in Colab

print("--- Dictionary Creation and Access ---")
# Creating a dictionary
student_profile = {
    "name": "John Doe",
    "age": 21,
    "major": "Computer Science",
    "gpa": 3.7,
    "courses": ["Python 101", "Data Structures", "Calculus I"]
}
print(f"Student Profile: {student_profile}")

# Accessing values
print(f"Student's Name: {student_profile['name']}")
print(f"Student's Age: {student_profile.get('age')}") # Using get()
print(f"Student's City (using get with default): {student_profile.get('city', 'Not Specified')}")

# Accessing a non-existent key will raise KeyError if not using .get()
# print(student_profile['city']) # Uncomment to see error

print("\n--- Adding and Modifying Dictionary Elements ---")
# Adding a new key-value pair
student_profile["city"] = "Bangkok"
print(f"After adding city: {student_profile}")

# Modifying an existing value
student_profile["gpa"] = 3.9
```

```
print(f"After updating GPA: {student_profile}")

# Using a variable as a key
favorite_language = "favorite_lang"
student_profile[favorite_language] = "Python"
print(f"After adding favorite language: {student_profile}")

print("\n--- Removing Elements from Dictionary ---")
# Using del
del student_profile["courses"]
print(f"After deleting 'courses': {student_profile}")

# Using pop()
removed_major = student_profile.pop("major")
print(f"Removed major: {removed_major}")
print(f"After popping 'major': {student_profile}")

# Using popitem() (removes last inserted in Python 3.7+ or arbitrary)
random_item = student_profile.popitem()
print(f"Removed random item: {random_item}")
print(f"After popitem: {student_profile}")

# student_profile.clear() # Empties the dictionary
# print(f"After clear: {student_profile}")

print("\n--- Iterating Through Dictionaries ---")
my_book = {
    "title": "The Python Journey",
    "author": "A. Coder",
    "pages": 350,
    "published": 2023
}

print("Iterating over keys:")
for key in my_book: # Default iteration is over keys
    print(key)

print("\nIterating over values:")
for value in my_book.values():
    print(value)
```

```python
print("\nIterating over key-value pairs (items):")
for key, value in my_book.items():
    print(f"{key}: {value}")

print("\n--- Checking for key existence ---")
if "author" in my_book:
    print("Author exists in the book dictionary.")
```

2. Sets: Creation, Modification, Operations

VS Code (set_operations.py):

Python

```python
# set_operations.py

print("--- Set Creation and Basic Operations ---")
# Creating a set from a list (duplicates are removed)
my_list_with_duplicates = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = set(my_list_with_duplicates)
print(f"Original list: {my_list_with_duplicates}")
print(f"Set from list (duplicates removed): {unique_numbers}")

# Creating a set directly
vowels = {"a", "e", "i", "o", "u"}
print(f"Vowels set: {vowels}")

# Important: To create an empty set, use set()
empty_set = set()
print(f"Empty set: {empty_set}, Type: {type(empty_set)}")
# Note: {} creates an empty dictionary, not an empty set

print("\n--- Adding and Removing Elements from a Set ---")
vowels.add("y") # Add an element
print(f"After adding 'y': {vowels}")

vowels.add("a") # Adding an existing element has no effect
print(f"After adding 'a' again: {vowels}")
```

```
vowels.remove("y") # Remove an existing element
print(f"After removing 'y': {vowels}")

try:
    vowels.remove("z") # Removing a non-existent element with remove() causes an error
except KeyError as e:
    print(f"Error trying to remove 'z' with remove(): {e}")

vowels.discard("z") # discard() does not raise an error if element not found
print(f"After trying to discard 'z': {vowels}")

# Pop an arbitrary element (sets are unordered)
popped_item = vowels.pop()
print(f"Popped item: {popped_item}")
print(f"Set after pop: {vowels}")

# vowels.clear() # Empties the set
# print(f"After clear: {vowels}")

print("\n--- Set Mathematical Operations ---")
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

print(f"Set A: {set_a}")
print(f"Set B: {set_b}")

# Union: all unique elements from both sets
print(f"Union (A | B): {set_a.union(set_b)}")
print(f"Union (A | B) operator: {set_a | set_b}")

# Intersection: common elements in both sets
print(f"Intersection (A & B): {set_a.intersection(set_b)}")
print(f"Intersection (A & B) operator: {set_a & set_b}")

# Difference: elements in A but not in B
print(f"Difference (A - B): {set_a.difference(set_b)}")
print(f"Difference (A - B) operator: {set_a - set_b}")

# Symmetric Difference: elements in either set, but not in both
print(f"Symmetric Difference (A ^ B): {set_a.symmetric_difference(set_b)}")
print(f"Symmetric Difference (A ^ B) operator: {set_a ^ set_b}")
```

```
print("\n--- Membership Testing and Subset/Superset ---")
my_skills = {"Python", "SQL", "Git", "Cloud"}
required_skills = {"Python", "SQL"}
extra_skills = {"Linux", "Python", "SQL", "Cloud", "API"}

print(f"'SQL' in my_skills: {'SQL' in my_skills}") # True
print(f"'Java' in my_skills: {'Java' in my_skills}") # False

print(f"Is required_skills a subset of my_skills? {required_skills.issubset(my_skills)}") # True
print(f"Is my_skills a superset of required_skills? {my_skills.issuperset(required_skills)}") # True
print(f"Are my_skills and extra_skills disjoint? {my_skills.isdisjoint(extra_skills)}") # False (they share elements)
```

Google Colab (in a code cell):

Python

```
# set_operations in Colab

print("--- Set Creation and Basic Operations ---")
# Creating a set from a list (duplicates are removed)
my_list_with_duplicates = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = set(my_list_with_duplicates)
print(f"Original list: {my_list_with_duplicates}")
print(f"Set from list (duplicates removed): {unique_numbers}")

# Creating a set directly
vowels = {"a", "e", "i", "o", "u"}
print(f"Vowels set: {vowels}")

# Important: To create an empty set, use set()
empty_set = set()
print(f"Empty set: {empty_set}, Type: {type(empty_set)}")
# Note: {} creates an empty dictionary, not an empty set

print("\n--- Adding and Removing Elements from a Set ---")
vowels.add("y") # Add an element
```

```
print(f"After adding 'y': {vowels}")

vowels.add("a") # Adding an existing element has no effect
print(f"After adding 'a' again: {vowels}")

vowels.remove("y") # Remove an existing element
print(f"After removing 'y': {vowels}")

try:
    vowels.remove("z") # Removing a non-existent element with remove() causes an error
except KeyError as e:
    print(f"Error trying to remove 'z' with remove(): {e}")

vowels.discard("z") # discard() does not raise an error if element not found
print(f"After trying to discard 'z': {vowels}")

# Pop an arbitrary element (sets are unordered)
popped_item = vowels.pop()
print(f"Popped item: {popped_item}")
print(f"Set after pop: {vowels}")

# vowels.clear() # Empties the set
# print(f"After clear: {vowels}")

print("\n--- Set Mathematical Operations ---")
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

print(f"Set A: {set_a}")
print(f"Set B: {set_b}")

# Union: all unique elements from both sets
print(f"Union (A | B): {set_a.union(set_b)}")
print(f"Union (A | B) operator: {set_a | set_b}")

# Intersection: common elements in both sets
print(f"Intersection (A & B): {set_a.intersection(set_b)}")
print(f"Intersection (A & B) operator: {set_a & set_b}")

# Difference: elements in A but not in B
print(f"Difference (A - B): {set_a.difference(set_b)}")
```

```
print(f"Difference (A - B) operator: {set_a - set_b}")

# Symmetric Difference: elements in either set, but not in both
print(f"Symmetric Difference (A ^ B): {set_a.symmetric_difference(set_b)}")
print(f"Symmetric Difference (A ^ B) operator: {set_a ^ set_b}")

print("\n--- Membership Testing and Subset/Superset ---")
my_skills = {"Python", "SQL", "Git", "Cloud"}
required_skills = {"Python", "SQL"}
extra_skills = {"Linux", "Python", "SQL", "Cloud", "API"}

print(f"'SQL' in my_skills: {'SQL' in my_skills}")
print(f"'Java' in my_skills: {'Java' in my_skills}")

print(f"Is required_skills a subset of my_skills? {required_skills.issubset(my_skills)}")
print(f"Is my_skills a superset of required_skills? {my_skills.issuperset(required_skills)}")
print(f"Are my_skills and extra_skills disjoint? {my_skills.isdisjoint(extra_skills)}")
```

แหล่งที่มา

1. https://gist.github.com/Denilson-Semedo/5e8c936b73025237d692dee296e1bf8f
2. https://github.com/SarthakBansal28062004/Python